

At-least-once 설계 원리

두 저장소 문제 · 처리 순서 불변 규칙 · 멍등성
강의 55min + 실습 30min

Redis Streams

At-least-once

ConsumerGroupWorker

5-Layer 아키텍처 리마인드

ARCHITECTURE REMINDER — 신뢰 경계로 시스템을 나눈다 · GATEWAY LAYER SEPARATES INTERNAL LAYERS



S9 핵심 구간: 레이어 2 **Redis Event Pipeline**에서 Consumer가 메시지를 어떻게 **안전하게 처리하는가** — At-least-once + 멍등성이 이 레이어를 지킨다.

두 저장소 문제 — 처리 완료를 어떻게 증명하는가

THE TWO-STORAGE PROBLEM

저장소 A — Redis PEL

XACK

"이 메시지는 내가
책임지고 처리했다"

PEL에서 메시지 제거
= 큐 시스템의 영수증

저장소 B — DB 원장

DB COMMIT

"실제 비즈니스 처리가
완료됐다"

user_nft_holdings +1
= 데이터 변경의 확정

핵심 문제 — 두 저장소는 독립적이다. DB 커밋과 Redis XACK를 하나의 원자적 트랜잭션으로 묶을 수 없다.

설계의 핵심 질문 — 두 저장소 중 하나가 반드시 "뒤쳐지는 순간"이 존재한다. 어느 쪽을 뒤쳐지게 할 것인가?

S5 복습 — Outbox 패턴 vs 오늘 배울 것

PRODUCER-SIDE vs CONSUMER-SIDE TWO-STORAGE PROBLEM

S5 복습 Outbox 패턴

발행 측(Producer) 두 저장소 문제

Webhook 수신

DB Outbox
(PENDING)

✱ queue.publish() 실패해도 DB에 증거 남음

Outbox Poller

Redis Streams
(Queue 진입)

보장하는 것 — 이벤트가 Queue에 반드시 들어간다

이어서



S9 오늘 At-least-once + 멍등성

소비 측(Consumer) 두 저장소 문제

Redis PEL
(Consumer 소유)

DB 원장
COMMIT

✱ DB 커밋 후 XACK 실패 → 재수신 → 중복 위험

멍등성 Guard

XACK
(절대 마지막)

보장하는 것 — Queue에서 꺼낸 이벤트가 정확히 1회 처리된다



두 패턴이 합쳐지면 E2E 신뢰성 완성 — S5 Outbox: "이벤트가 Queue에 반드시 들어간다" + S9 At-least-once: "Queue에서 꺼낸 이벤트가 반드시 처리된다" → 발신부터 원장 기록까지 유실 없음

이상적 세계 vs 실제 세계

IDEAL WORLD vs REALITY

이상적 세계 (존재하지 않음)

DB 커밋

+

Redis XACK

하나의 원자적 트랜잭션으로 묶임

→ 둘 다 성공 or 둘 다 실패

→ Exactly-once 보장

Redis가 XA 프로토콜을 지원하지 않아 불가

실제 세계

DB 커밋



crash

XACK 실행 안 됨

→ PEL에 메시지 잔류 → 재수신 → 중복 처리 위험

XACK



crash

DB 커밋 실행 안 됨

→ 메시지 PEL에서 사라짐 → 재수신 불가 → 데이터 영구 유실

결론 — Exactly-once = 이상 / At-least-once + 멍등성 = 현실적 정답

Exactly-once — 왜 2PC로 해결 못 하는가

WHY 2PC (TWO-PHASE COMMIT) DOESN'T WORK HERE

2PC란?

두 개 이상의 저장소에 걸친 트랜잭션을
코디네이터가 조율해 원자적으로 커밋

준비 단계 → 커밋 단계
= XA 프로토콜 기반



XA 미지원

Redis는 XA 프로토콜을 지원
하지 않음 → 분산 트랜잭션
코디네이터 불가



복잡도 폭발

구현 난이도 급상승 코디네
이터 장애 시 복구가 더 어려
워짐



성능 저하

코디네이터 병목 네트워크
왕복 2회 처리량 대폭 감소



Exactly-once

VS



At-least-once + 멍등성

중복 수신은 허용하되, **멍등성**으로 중복 처리를 차단한다.
단순하고, 장애에 강하고, 확장 가능하다.

DB COMMIT vs Redis XACK — 무엇이 다른가

TWO DIFFERENT KINDS OF "COMMIT"

DB COMMIT

확정 대상	데이터 변경 (insert/update/delete)
물어보는 질문	"이 데이터 변경을 영속화할까?"
위치	DB 트랜잭션의 종결자
실패 시	전체 롤백 (원자성 보장)

Redis XACK

확정 대상	메시지 처리 완료 표시
물어보는 질문	"이 메시지 책임을 내가 다 졌나?"
위치	큐 시스템의 영수증
실패 시	PEL 잔류 → 재수신 가능

같은 점 — 둘 다 "확정"하는 동작

다른 점 — 무엇을 확정하는지가 완전히 다름. DB는 *데이터*, XACK는 *메시지 소유권*을 확정한다.

비유 — 식당 계산과 영수증

ANALOGY: CARD PAYMENT vs RECEIPT



방법 1 — 영수증 먼저

= XACK 먼저, DB 처리 나중

❌ 위험

영수증 발행

→

카드 단말기 오류 ✖

영수증은 있는데 돈은 안 빠져나감

→ 식당 손해 = 데이터 유실



방법 2 — 카드 승인 먼저

= DB 커밋 먼저, XACK 나중

✅ 안전

카드 승인 ✅

→

영수증 프린터 오류 ✖

손님 재요청 → 영수증 재발행 가능

최악이어도 중복 발행 → 멍등성으로 차단

결론 — 카드 승인(DB 커밋) 먼저, 영수증(XACK) 나중이 안전하다. 중복은 멍등성이 막는다.

잘못된 순서 ① — XACK 먼저 (치명적)

WRONG ORDER: ACK BEFORE PROCESSING — DATA LOSS

1

XREADGROUP → PEL 소유 취득



2

XACK ← PEL에서 제거 (처리 완료 선언)



3

✱ DB 처리 실패 — 여기서 장애



4

재시작해도 PEL에 메시지 없음 → 영구 유실

왜 치명적인가

XACK = "처리 완료" 선언

→ Redis PEL에서 메시지 삭제

→ 재시작해도 재수신 불가

→ 원장 업데이트 누락

→ 블록체인에는 NFT 있는데

원장에는 없음

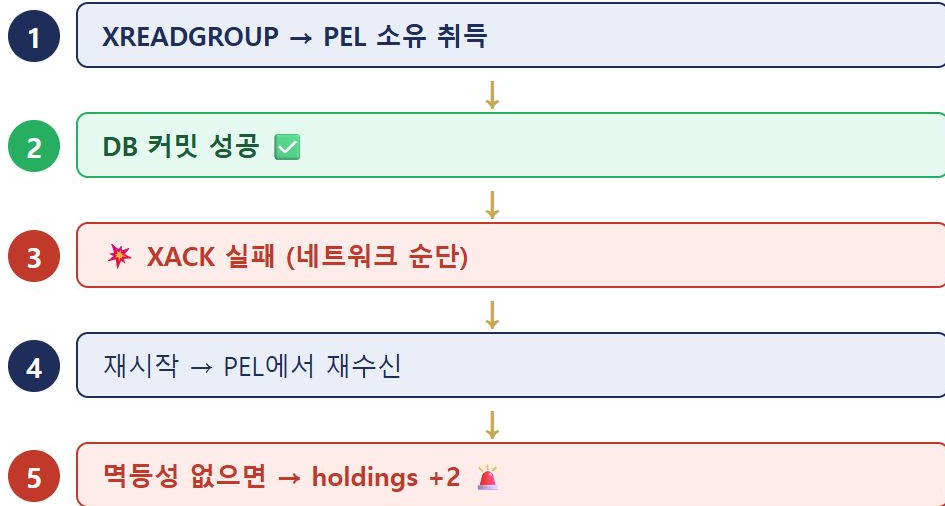
→ 회계 불일치 = 감사 실패

복구 방법이 없다

메시지가 PEL에서 사라졌으므로 어떤 재시도도 불가능

잘못된 순서 ② — DB 먼저, 멱등성 없음

WRONG ORDER: DB FIRST WITHOUT IDEMPOTENCY — DUPLICATE



멱등성이 있다면?

- ✓ 재수신 → 이미 처리됨 감지
- ✓ DB 처리 건너뛴
- ✓ XACK만 재실행 → 정합성 유지

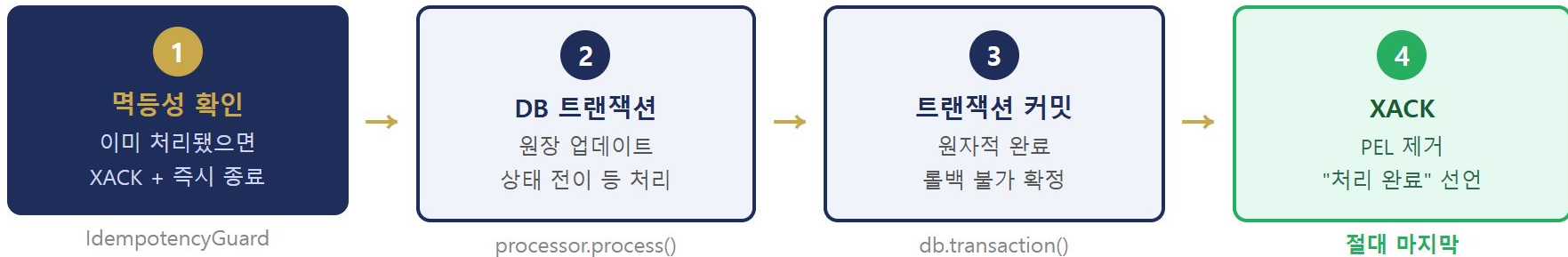
멱등성이 없다면?

DB 커밋 → XACK 실패 → 재수신
→ DB 커밋 또다시 → 원장 +2
→ NFT 이중 지급 = 금융 사고

DB 먼저 + XACK 나중은 멱등성이 있을 때만 안전하다

올바른 처리 순서 — 4단계 불변 규칙

THE FOUR-STEP INVARIANT — NEVER CHANGE THIS ORDER



불변 규칙 — XACK는 반드시 Step 4. DB 커밋 성공을 확인한 후에만 실행한다. 이 순서는 절대 바꾸지 않는다.

각 단계가 그 위치에 있는 이유

WHY EACH STEP IS WHERE IT IS

1 멍등성 — 가장 먼저

중복 수신 시
DB 접근조차 없이
즉시 차단

비용: XACK 1회만
효과: DB 중복 방지

2 DB — 원자성 보장

원장 업데이트 +
상태 전이 등을
하나의 트랜잭션으로

중간 장애 → 전체 롤백
부분 커밋 없음

4 XACK — 절대 마지막

XACK 전 크래시
→ PEL 잔류
→ 재수신 가능

멍등성이 재수신 후
중복 처리를 차단

XACK 실패 시 복구 흐름 — 재시작 → XREADGROUP/XAUTOCLAIM으로 PEL 재수신 → 멍등성: 이미 DB 커밋됨 감지 → XACK만 재실행 (DB 처리 없이) → 정합성 유지 ✓

크래시 A — 멍등성 확인 중 장애

CRASH SCENARIO A: BEFORE DB TRANSACTION



크래시 A 결과 분석

DB 변경 없음 — 처리 안 됨
PEL 메시지 존재 — XACK 없음

복구 흐름

재시작 → 재수신
→ 멍등성 통과 (미처리)
→ 정상 처리 → XACK

At-least-once ☒

크래시 B — DB 트랜잭션 중 장애

CRASH SCENARIO B: DURING DB TRANSACTION (ROLLBACK)



크래시 B 결과 분석

DB 변경 없음 — 트랜잭션 롤백

PEL 메시지 존재 — XACK 없음

복구 흐름

재시작 → 재수신

→ 멍등성: 아직 미처리

→ DB 재처리 → XACK

At-least-once ✓

DB 트랜잭션 롤백 = 멍등성 키도 미저장 → 재처리 가능

크래시 C — XACK 직전 장애 (가장 흔한 케이스)

CRASH SCENARIO C: DB COMMITTED, XACK NOT YET — SAFE WITH IDEMPOTENCY



크래시 C 결과 분석

DB 변경 완료 — 커밋됨 + 멍등성 키 저장

PEL 메시지 존재 — XACK 실패

복구 흐름

재수신 → 멍등성: 이미 처리됨

→ DB 처리 건너뛰

→ XACK만 재실행

중복 없이 At-least-once ✓

모든 크래시 케이스 — 정합성 유지 요약

ALL CRASH SCENARIOS — CONSISTENCY GUARANTEED

케이스	크래시 지점	DB 상태	PEL 상태	재시작 후 복구
크래시 A	먹등성 확인 중	변경 없음	메시지 잔류	재수신 → 먹등성 통과 → 재처리 ☒
크래시 B	DB 트랜잭션 중	롤백됨	메시지 잔류	재수신 → 먹등성: 미처리 → 재처리 ☒
크래시 C	XACK 직전	커밋 완료	메시지 잔류	재수신 → 먹등성: 이미처리 → XACK만 ☒

모든 크래시 지점에서 정합성 유지



= At-least-once + 먹등성의 힘

PEL이 "미처리 메시지 보험"이고, 먹등성이 "중복 처리 방화벽"이다.

코드로 보는 4단계 — _handleWithRetry()

ConsumerGroupWorker.ts — THE FOUR-STEP INVARIANT IN CODE

```
private async _handleWithRetry(msg: StreamMessage): Promise<void> {
  const retryCount = parseInt(msg.fields['_retryCount'] ?? '0', 10);

  // — ① 멍등성 확인 + ② DB 트랜잭션 + ③ 커밋 —————
  // processor.process() 내부에서 IdempotencyGuard.run() + db.transaction() 처리
  try {
    await Promise.all(
      this.processors
        .filter(p => p.eventTypes.includes(msg.fields['eventType'] ?? ''))
        .map(p => p.process(msg))
    );

    // — ④ XACK - 절대 마지막 —————
    await this.redis.xack(this.config.streamKey, this.config.groupName, msg.id);

  } catch (err) {
    // DB 실패 → XACK 실행하지 않음 → PEL 잔류 → 재시도
    msg.fields['_retryCount'] = String(retryCount + 1);
  }
}
```

코드로 보는 4단계 — 단계별 해설

ConsumerGroupWorker.ts — WHAT EACH STEP DOES INTERNALLY

① ② ③ — `processor.process()` 내부

- ① `IdempotencyGuard.run()` 호출
→ 이미 처리된 requestId면 callback 건너뛴
- ② `db.transaction(trx => { ... })`
→ `user_nft_holdings` INSERT
→ `ON CONFLICT DO NOTHING` (멥등성)
- ③ 트랜잭션 커밋 (자동)
→ 원자적 완료 확정

④ XACK — 절대 마지막

- try 성공 후에만 실행
- PEL에서 메시지 제거
- "처리 완료" 공식 선언

catch — XACK 없음

- PEL 잔류 → `minIdleMs` 후 재수신
- `_retryCount` 누적
- `maxRetries` 초과 시 DLQ

단계 위반 시 — 실제로 어떤 일이 일어나는가

WHAT HAPPENS WHEN THE INVARIANT IS VIOLATED

❌ 위반 ① — 멍등성 없이 DB 먼저

DB 커밋 → XACK → 재수신 → DB 커밋

user_nft_holdings amount = 2, 3, 4...

→ NFT 이중 지급 = 금융 사고

❌ 위반 ② — XACK를 Step 1보다 먼저

XACK → DB 실패 → PEL 없음 → 영구 유실

원장 누락 → 블록체인에는 NFT

DB에는 없음 → 회계 불일치 = 감사 실패



규칙은 하나 — 멍등성 확인 → DB 커밋 → XACK. 이 순서를 바꾸면 금융 데이터 정합성이 깨진다.

Consumer 수평 확장과 멱등성

HORIZONTAL SCALING — IDEMPOTENCY MAKES IT SAFE

Consumer Group 메시지 분배 방식

Consumer-1

메시지 A, D, G

Consumer-2

메시지 B, E, H

Consumer-3

메시지 C, F, I

XREADGROUP: 먼저 호출한 Consumer가 메시지 소유

→ 같은 메시지를 두 Consumer가 동시에 처리 불가 (정상 상황)

XAUTOCLAIM 예외 상황

Consumer-1 장애 → 메시지 A PEL 잔류

→ Consumer-2가 XAUTOCLAIM으로 탈취

→ 멱등성 확인 → 이미 처리? 아니면 재처리

→ 어느 쪽이든 정합성 유지

멱등성 키

온체인: txHash + logIndex

Webhook: requestId

DB: processed_events UNIQUE

→ ON CONFLICT DO NOTHING

→ 중복 삽입 시 조용히 무시

결론 — Consumer를 N배 늘려도 멱등성이 중복 처리를 차단한다. 수평 확장과 정합성이 공존한다.

핵심 암기 + S09 실습 안내

KEY TAKEAWAYS + EXERCISE GUIDE

핵심 암기 — 이것만 기억하라

- 1 멍등성 확인 먼저
- 2 DB 트랜잭션 커밋
- 3 XACK — 절대 마지막

중복은 멍등성이 막고
유실은 XACK 후치(後置)가 막는다

S09 실습 — S09_atleastonce.ts

- Part 1** 동일 메시지 2회 처리 → 원장 +2 (버그 재현)
- Part 2** IdempotentNftProcessor 구현 (TODO 1~5)
- 검증** Set<string> 기반 인메모리 멍등성

실행 — 프로젝트 루트에서

```
npm run exercise:s09
```

S09 → S10 연결 — S09: 4단계 규칙 설계 이해 → S10: ConsumerGroupWorker 전체 구현 (_handleWithRetry, _reclaimPending, DLQ 연동)